INSTITUTO FEDERAL DE SÃO PAULO — CAMPUS HORTOLÂNDIA

PROPOSTA DE ESTEIRA DEVSECOPS COM SAST, DAST, SCA E IA GENERATIVA

Projeto: Entre a Tecnologia e a Sustentabilidade — os Desafios Ambientais na Implantação de Data Centers

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas
Disciplina: Práticas em Sociedade e Informática 3
Professor: Carlos Eduardo Pagani
Integrantes: Isabele Campos de Oliveira, Maravania de Paula Souza e Pietra Caracco Ruiz

Hortolândia – SP | 2026

1. Resumo executivo

Propõe-se uma esteira de integração e entrega contínuas (CI/CD) com controles de segurança incorporados ao desenvolvimento (DevSecOps), combinando testes automatizados, análise estática (SAST), análise dinâmica (DAST), análise de dependências (SCA) e uso supervisionado de IA generativa. O objetivo é reduzir retrabalho, detectar vulnerabilidades antes da implantação e preservar a confiabilidade do dashboard educativo e do agente conversacional de análise ambiental.

A proposta é um plano de implantação, não uma declaração de ferramentas já configuradas. Os documentos-base descrevem arquitetura, requisitos e testes planejados; tecnologias específicas e limites operacionais abaixo são recomendações de implementação.

2. Fundamentação nos documentos do projeto

• Documento de planejamento: finalidade educativa, público acadêmico e geral, dados sobre energia, água, emissões, resíduos, recursos naturais e benefícios; responsabilidade da equipe pelos testes do dashboard.

• Matriz de requisitos e arquitetura: RF01–RF12, RNF01–RNF08, arquitetura em camadas (dashboard, serviços, dados), tabelas DATA_CENTER, INDICADOR_AMBIENTAL, BENEFICIO e FONTE, testes TS01–TS09 e ambientes de desenvolvimento, homologação e produção.

• Análise de viabilidade do agente: RF13–RF18, RNF09–RNF14, HU07/UC12, testes TS10–TS15, endpoint ilustrativo POST /api/ai/chat, proteção de chaves, resposta fundamentada em fontes e tratamento de falta de dados.

Para o MVP, a documentação recomenda consultas controladas aos dados internos, sem pesquisa irrestrita na Internet. Caso futuramente seja habilitada pesquisa externa, ela deverá constituir uma evolução separada, com testes de origem, autorização e confiabilidade das fontes.

3. Arquitetura técnica considerada

Frontend web (sugestão: React/Vite) → API REST (sugestão compatível com a implementação: Java/Spring Boot) → serviços de indicadores, mapa, comparações e agente → PostgreSQL. O serviço do agente consulta dados autorizados e suas fontes antes de chamar o modelo de linguagem; credenciais permanecem no backend. Docker pode padronizar o ambiente de testes, sem exigir Kubernetes no MVP.

Os componentes React/Vite, Spring Boot e PostgreSQL refletem o contexto de implementação informado durante o desenvolvimento, mas os documentos anexados não comprovam sua adoção definitiva. Confirmar a estrutura real do repositório antes de ativar os jobs.

4. Desenho da esteira CI/CD

Fase / gatilho	Automação proposta	Condição de aprovação
0. Issue e branch	Especificação SDD, critérios de aceite, branch feature/* e pull request.	Requisito e testes associados identificados.
1. Validação rápida — push/PR	Formatação, lint, compilação, verificação de segredos e testes unitários.	Build e verificações obrigatórias aprovados.

Fase / gatilho	Automação proposta	Condição de aprovação
2. SAST e SCA — PR	Semgrep ou CodeQL; OWASP Dependency-Check/OSV-Scanner; análise do frontend e backend.	Sem achados críticos/altos novos sem exceção formal.
3. Integração — PR	Testcontainers/PostgreSQL isolado, testes de API e contratos; fixtures sintéticas.	Contratos, persistência e regras de negócio corretos.
4. Pacote — merge develop	Build de artefatos e imagem Docker, SBOM, scan de imagem com Trivy.	Artefato rastreável e sem risco bloqueante.
5. Homologação — deploy	Ambiente efêmero ou staging; Playwright e ZAP baseline autenticado quando cabível.	Smoke, fluxos críticos e DAST aprovados.
6. Produção — tag/release	Aprovação humana, deploy controlado, health check e rollback.	Checklist de liberação assinado; observabilidade ativa.

Fluxo: commit → PR → lint/segredos/unitários → SAST + SCA → integração → build/SBOM/scan de imagem → homologação → DAST + E2E → aprovação → produção → monitoramento. Jobs independentes de PR podem executar em paralelo; o DAST exige uma aplicação em execução.

5. SAST, SCA e DAST: objetivos e ferramentas

Controle	O que detecta / escopo	Ferramentas candidatas
SAST	Padrões de código vulneráveis: injeção, validação insuficiente, configuração insegura e uso indevido de APIs.	Semgrep CE; CodeQL quando compatível com a licença e o repositório.
SCA	Dependências com CVEs, componentes obsoletos, licenças e inventário SBOM.	OWASP Dependency-Check, OSV-Scanner, Dependabot ou Renovate; CycloneDX.
DAST	Falhas observáveis em aplicação rodando: cabeçalhos, exposição, autenticação e entradas maliciosas.	OWASP ZAP Baseline no staging; scan mais profundo em janela controlada.
Complementares	Segredos, imagens de container, IaC e configurações.	Gitleaks; Trivy; Checkov se houver infraestrutura como código.

SAST, SCA e DAST são complementares, não substitutos. Um scanner não demonstra ausência de vulnerabilidades. Achados precisam de triagem, avaliação de explorabilidade e registro de falsos positivos.

6. Política de bloqueio e tratamento de vulnerabilidades

• PR: bloquear falhas de build, testes críticos, segredos confirmados e achados SAST/SCA críticos ou altos novos e relevantes; permitir exceção somente com justificativa, responsável e prazo.

• Homologação: bloquear falhas DAST de alto risco confirmadas e falhas E2E nos fluxos principais; alertas médios devem ser triados conforme impacto e exposição.

• Produção: exigir aprovação humana, versão identificada, registro de testes e plano de rollback. Não implantar automaticamente após falha de controle obrigatório.

• Evitar divulgar credenciais em logs, relatórios, artefatos, prompts ou mensagens ao ChatHub; usar segredos no gerenciador do provedor CI.

7. Plano de testes por componente

Componente	Cenários automatizados prioritários	Ferramentas
Frontend	Gráficos, filtros, responsividade, estados de erro, links de fontes e navegação no mapa.	Vitest + Testing Library; Playwright.
API Spring Boot	Validação de parâmetros, paginação, status HTTP, serialização, erros e autorização.	JUnit 5, Mockito, Spring Boot Test, REST Assured.
Banco de dados	Relacionamentos DATA_CENTER/INDICADOR_AMBIENTAL/FONTE, migrações, unidades e períodos.	Testcontainers PostgreSQL + Flyway/Liquibase se adotado.
Agente de IA	Consulta autorizada, fontes, resposta sem dados, ambiguidade, prompt injection, timeout e indisponibilidade.	JUnit com mock de provedor + conjunto de avaliações de respostas.
Segurança	Injeção, XSS, exposição de tokens, endpoints administrativos e permissões.	Semgrep, ZAP, Gitleaks, Trivy.

Casos específicos: o agente não deve inventar “Data Center mais impactante” sem indicador definido; comparações exigem unidades e períodos compatíveis; respostas devem citar fontes existentes e admitir insuficiência de dados. Os testes de IA devem verificar critérios semânticos e fatos, não coincidência literal de texto.

8. Matriz de rastreabilidade proposta

Requisitos	Casos / evidências	Porta de qualidade
RF01–RF12	TS01–TS09: dashboard, energia, água, emissões, comparação, filtros, fontes e mapa.	Unitários + API + E2E + acessibilidade.
RF13–RF17	TS10–TS14: pergunta, consulta, comparação, fonte e ausência de dados.	Integração + avaliações de IA + E2E.
RNF10–RNF11	TS15: segredo protegido e provedor indisponível.	SAST + secret scan + teste de resiliência.
RNF12–RNF14	Rastreabilidade, latência e restrição de escopo.	Avaliações de IA + métricas + testes adversariais.
RF18	Histórico restrito à sessão, sem persistência indevida.	Unitário + privacidade + E2E.

9. Integração da IA generativa e ChatHub no ciclo

ChatHub é considerado aqui uma interface para comparar respostas de diferentes modelos, e não um scanner SAST/DAST/SCA nem um componente obrigatório de execução automática no CI. A disponibilidade de modelos, planos, integrações e políticas de retenção deve ser verificada antes da

adoção.

Fluxo de uso com revisão humana

• 1. Preparar um pedido SDD: requisito, contexto mínimo, contrato da API, critérios de aceite e restrições de segurança, usando código sanitizado.

• 2. Enviar o mesmo problema a dois ou três modelos via ChatHub; solicitar diagnóstico, alternativas, riscos, testes propostos e justificativas.

• 3. Comparar soluções por correção, segurança, manutenibilidade, desempenho, esforço, custo e aderência à arquitetura.

• 4. Validar no ambiente local: executar testes, SAST/SCA, benchmark quando pertinente; nunca aceitar código só porque modelos concordaram.

• 5. Registrar decisão técnica (ADR curta) com alternativa escolhida, evidências e revisor; incorporar testes de regressão.

Exemplo aplicado

Problema: POST /api/ai/chat apresenta timeout e respostas sem fontes. Alternativa A: aumentar timeout (rápida, não corrige causa); B: reduzir e validar contexto, cachear consultas não sensíveis e impor limite de tokens (preferencial para o MVP após medição); C: fila assíncrona (mais complexa, adequada apenas se houver demanda comprovada). Solicitar ao ChatHub testes de timeout, fontes obrigatórias e fallback; comparar as propostas e confirmar com medições.

10. Matriz de decisão de alternativas

Pontuações abaixo são estimativas exemplificativas (1 = pior, 5 = melhor), não benchmarks medidos. Pesos: aderência 30%, facilidade 25%, custo 20%, segurança/controle 25%.

Opção	Aderência	Facilidade	Custo	Controle	Total / 5
GitHub Actions + Semgrep + OSV + ZAP	5	4	4	4	4,30
Azure Pipelines + mesmas ferramentas	5	3	4	4	4,05
Jenkins autogerenciado + ferramentas	4	2	3	4	3,30

Recomendação condicional: GitHub Actions se o código estiver no GitHub e houver cota suficiente; Azure Pipelines se o repositório estiver no Azure DevOps; Jenkins apenas se a equipe puder manter agentes e infraestrutura. As ferramentas de segurança podem ser reutilizadas nas três opções. Confirmar políticas de licenciamento, custos e limites antes da decisão.

11. Exemplo de pipeline em GitHub Actions (referência adaptável)

Estrutura lógica sugerida, sem presumir que os scripts já existam:

• Gatilhos: pull_request em develop/main; push em develop; release por tag.

• Job frontend: setup Node + pnpm, pnpm install --frozen-lockfile, pnpm lint, pnpm test, pnpm build.

• Job backend: setup Java, ./mvnw -B verify; testes de integração com PostgreSQL isolado.

• Job segurança: Gitleaks; Semgrep; OSV-Scanner/Dependency-Check; produzir relatórios e SBOM.

• Job imagem: docker build e Trivy; publicar imagem apenas se gates aprovados.

• Job staging: deploy com variáveis protegidas; smoke e Playwright; OWASP ZAP baseline na URL autorizada.

• Job release: aprovação de ambiente, deploy de tag, health check e rollback se necessário.

Ajustar caminhos frontend/backend, versões do Java/Node, scripts, tokens, licenças e política de forks ao repositório real. Em pull requests externos, não disponibilizar segredos de produção. DAST só deve atingir instâncias de teste controladas, nunca sistemas de terceiros.

12. Metas e indicadores de acompanhamento

Indicador	Como medir	Meta inicial proposta
Tempo de feedback do PR	Minutos do push até jobs rápidos concluídos.	Até 10 min (ajustar após baseline).
Taxa de aprovação da esteira	Pipelines aprovados / pipelines executados.	Acompanhar tendência semanal.
Cobertura de testes	Cobertura de linhas/branches e cenários críticos.	≥ 80% nos serviços críticos, sem sacrificar qualidade.
Vulnerabilidades abertas	Achados confirmados por severidade e idade.	0 críticas/altas novas sem tratamento no release.
Qualidade do agente	Casos corretos, com fonte válida e sem invenção / total avaliado.	≥ 95% no conjunto de avaliação do MVP.
Lead time de mudança	Tempo do commit até homologação/produção.	Reduzir após medição inicial.
Taxa de falha de mudança	Releases com rollback/correção urgente / releases.	Acompanhar e reduzir.

As metas são sugestões de partida e não resultados observados. Medir uma linha de base nas primeiras execuções e renegociar limites com a equipe.

13. Cronograma de adoção (quatro semanas, estimativa)

Semana	Entregáveis	Critério de conclusão
1	Padronização do repositório, scripts de teste, lint, secret scan, PR template e CI básica.	Build reproduzível e testes rápidos em cada PR.
2	SAST, SCA, SBOM, testes de integração e regras de bloqueio.	Relatórios acessíveis e vulnerabilidades triadas.
3	Staging, E2E, ZAP baseline, smoke, métricas e rollback.	Fluxos críticos aprovados em ambiente isolado.

Semana	Entregáveis	Critério de conclusão
4	Avaliação do agente, protocolo ChatHub, ADRs, documentação e ensaio da SNCT.	Checklist de release e demonstração reproduzível.

14. Governança, papéis e riscos

A matriz RACI original atribui testes do dashboard a Pietra, desenvolvimento do dashboard a Isabele e Maravania, e aprovação/consulta à equipe e ao professor conforme a atividade. Para a esteira, propõe-se que a responsável por testes coordene a definição de casos e evidências; as desenvolvedoras corrijam achados e revisem PRs; e a equipe aprove releases. Esta divisão é uma proposta, não uma nova atribuição já acordada.

• Falsos positivos: triagem documentada, supressões justificadas e prazo de reavaliação.

• Exposição de dados a modelos externos: anonimizar exemplos, proibir segredos e dados pessoais; verificar termos de retenção e políticas institucionais.

• Dependência do modelo: testes determinísticos para regras de negócio, mocks no CI e avaliações periódicas com conjunto fixo de perguntas.

• Prazo curto do MVP: priorizar lint, unitários, SAST, SCA, secret scan, E2E crítico e ZAP baseline; aprofundar segurança em ciclos posteriores.

• Dependência de serviços externos: limites de custo, timeout, fallback e dashboard disponível mesmo quando o modelo falhar.

15. Critérios de aceite e conclusão

A esteira estará pronta para uso inicial quando: (a) todo PR executar build, testes e scanners; (b) resultados ficarem registrados e revisáveis; (c) homologação permitir executar E2E e DAST sem atingir produção; (d) segredos não forem expostos; (e) respostas do agente forem avaliadas quanto a fonte, escopo e falta de dados; (f) existir aprovação e rollback de release; (g) a equipe demonstrar uma correção assistida por ChatHub, comparar alternativas e registrar a decisão.

A solução proposta amplia a estratégia de testes já prevista nos documentos do projeto e integra segurança ao ciclo de desenvolvimento. O ganho esperado é feedback mais cedo, menor retrabalho e maior confiabilidade, sem delegar à IA generativa decisões de segurança ou aprovação de código.

Referências e bases consultadas

• OLIVEIRA, Isabele Campos de; SOUZA, Maravania de Paula; RUIZ, Pietra Caracco. Entre a Tecnologia e a Sustentabilidade: os Desafios Ambientais na Implantação de Data Centers. Documento parcial, IFSP, 2026.

• Matriz de Requisitos e Modelagem da Arquitetura de Software. Projeto Data Centers, IFSP, 2026.

• Análise de Viabilidade e Impacto da Introdução de Agente de Inteligência Artificial. Projeto Data Centers, IFSP, 2026.

• OWASP. Web Security Testing Guide; Zed Attack Proxy (ZAP); Dependency-Check. Documentação oficial.

- Semgrep. Documentação de análise estática. https://semgrep.dev/docs/
- Google. OSV-Scanner. https://google.github.io/osv-scanner/
- GitHub. GitHub Actions Documentation. https://docs.github.com/actions
- OWASP. Top 10 for Large Language Model Applications. Documentação de referência.